



TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Problemas NP-Completo

[illegible]

12 de junio de 2026

Alejandro, Pablo Martin
98021

Prada, Matias Leonel
99397

Poli Salvador, Ignacio Manuel
110399

1. Consideraciones

Este documento está pensado para que sirva de guía, e incluso template a la hora de realizar un informe para la materia (por no decir, cualquier documento formal en la facultad). La idea es utilizar $\text{\LaTeX}$, pura y exclusivamente para que tenga un formato acorde. Es decir, no hecho en un simple procesador de texto (como pudiese ser Word).

Aquí sólo se mostraran algunas de las cosas que se pueden hacer con $\text{\LaTeX}$. Si se quiere hacer algo específico, o de otra forma, abunda la documentación (y artículos de StackOverflow) al respecto.

Definimos ciertos lineamientos técnicos a considerar para los informes:

- En el informe debe estar incluido el código más importante (es decir, los algoritmos, no un main) cuando corresponda.
- Debe estar explicado cómo se hacen los sets de prueba. Inclusive, si notan alguna falencia con esta generación que pueda afectar de alguna forma el análisis del resultado, se espera que lo enuncien (hay trabajos donde esto no sucederá, pero hay trabajos donde sí, especialmente cuando vemos aproximaciones).
- Los gráficos deben estar bien generados: deben tener título, los ejes deben tener tanto qué refieren como qué unidad de medida usan (en caso de tenerla). En caso de tener gráficos superpuestos, debe quedar claro qué es cada gráfico.
- **Importante:** en caso de tener que realizar una reentrega (y por ende, tener que cambiar cosas del informe), salvo que haya que rehacer una enorme cantidad del mismo, no modificar el informe anterior sino agregar un anexo al final indicando las correcciones realizadas en la nueva entrega, para agilizar las correcciones.

A su vez, dejamos algunas breves líneas sobre cuestiones de redacción:

- Evitar errores ortográficos y gramaticales.
- El lenguaje debe ser técnico. No tiene que por eso ser un texto aburrido, pero evitar frases como *"la cosa que más me complicó fue..."*, *"creo que..."*.
- Usar plural de modestia, incluso si el trabajo fuera realizado por una única persona.

2. Análisis del Problema

El presente trabajo plantea el desafío de seleccionar un conjunto de jugadores para disputar un partido amistoso con el objetivo estratégico de apaciguar las presiones de la prensa. Para lograrlo, el equipo técnico debe asegurarse de contentar a diferentes medios periodísticos, seleccionando al menos un jugador del agrado de cada uno de ellos, pero comprometiendo la menor cantidad de cupos posibles en el equipo.

Formalizando el escenario, podemos definir los elementos del problema de la siguiente manera:

- Un conjunto universal A conformado por los n jugadores convocados (en este caso, los 43 jugadores de la lista).
- Una serie de m subconjuntos $B_1, B_2, \dots, B_m$, donde cada $B_i \subseteq A$ representa el subconjunto de jugadores que exige ver en cancha el i -ésimo medio de prensa.

El objetivo original del problema consiste en encontrar un conjunto $C \subseteq A$ de tamaño mínimo tal que logre intersectar a todos los subconjuntos de preferencias periodísticas. Es decir, se debe garantizar la siguiente condición:

$$C \cap B_i \neq \emptyset \quad \forall i \in \{1, 2, \dots, m\} \quad (1)$$

Este escenario modela un problema clásico de la teoría de la complejidad computacional y optimización combinatoria conocido como el **Hitting-Set Problem**.

Sin embargo, para los fines de las demostraciones teóricas iniciales de este trabajo práctico (demostración de pertenencia a NP y prueba de NP-Complejidad), es necesario abstraerse de la minimización del conjunto. Por lo tanto, abordaremos y utilizaremos estrictamente la **versión de decisión** del Hitting-Set Problem.

En esta versión, se introduce una cota superior k (que en nuestro dominio representa la cantidad máxima de jugadores que el técnico está dispuesto a "gastar" para contentar a la prensa) y el problema se reduce a responder afirmativa o negativamente a la siguiente pregunta:

Dado el conjunto A de n elementos, los m subconjuntos $B_1, B_2, \dots, B_m$ de A , y un número entero k , ¿existe un subconjunto $C \subseteq A$ con $|C| \leq k$ tal que C tenga al menos un elemento de cada B_i ?

3. Demostración de Pertenencia a NP

Para demostrar que la versión de decisión del *Hitting-Set Problem* pertenece a la clase NP, debemos probar que dada una solución candidata (un certificado), es posible verificar su validez en un tiempo acotado por un polinomio.

A continuación, se presenta el algoritmo certificador escrito en Python que valida una solución propuesta:

```
1 def verificador_hitting_set(k, solucion, subsets, universo):
2     """
3     Verifica en tiempo polinomial si una solucion candidata
4     es valida para el Hitting Set Problem (version decision).
5     """
6
7     # 1. Verificar restriccion de cardinalidad
8     if len(solucion) > k:
9         return False
10
11    # 2. Verificar que los elementos pertenezcan al universo
12    for elemento in solucion:
13        if elemento not in universo:
14            return False
15
16    # 3. Verificar condicion de interseccion (Hitting Set)
17    for subconjunto in subsets:
18        interseccion_encontrada = False
19
20        for elemento in solucion:
21            if elemento in subconjunto:
22                interseccion_encontrada = True
23                break # Globo pinchado, pasamos al siguiente
24
25        if not interseccion_encontrada:
26            return False
27
28    return True
```

Análisis de Complejidad

Para justificar que el validador opera en tiempo polinomial, definimos las variables que representan el tamaño de nuestra entrada:

- n : Cantidad de elementos en el universo ($|A|$).
- m : Cantidad de subconjuntos en `subsets`.
- c : Cantidad de elementos en la `solucion` candidata ($|C|$). Por definición lógica del problema, $c \leq n$.

Evaluamos el costo temporal de cada paso del algoritmo en el peor de los casos:

1. **Verificación de cardinalidad (Línea 8):** Obtener la longitud de la solución candidata y compararla con k toma un tiempo de $\mathcal{O}(1)$ (o a lo sumo $\mathcal{O}(c)$ dependiendo de la implementación subyacente de la estructura de datos).
2. **Pertenencia al universo (Líneas 12-14):** Se itera sobre los c elementos de la solución, verificando para cada uno si pertenece al universo de tamaño n . El costo en el peor caso es $\mathcal{O}(c \cdot n)$.
3. **Condición de Hitting Set (Líneas 17-26):** Se itera sobre los m subconjuntos. Para cada uno (que puede tener hasta n elementos), se busca si contiene alguno de los c elementos de la solución. Este proceso de búsquedas anidadas requiere a lo sumo $\mathcal{O}(m \cdot c \cdot n)$ operaciones.

El tiempo total de ejecución en el peor de los casos está acotado asintóticamente por el paso más costoso: $\mathcal{O}(m \cdot c \cdot n)$. Dado que la cantidad de elementos en la solución nunca será mayor al universo total ($c \leq n$), podemos simplificar la cota superior a $\mathcal{O}(m \cdot n^2)$.

Como la función $\mathcal{O}(m \cdot n^2)$ es un polinomio respecto al tamaño de la entrada, queda formalmente demostrado que la verificación de un certificado se puede realizar en tiempo polinomial. En conclusión, **el Hitting-Set Problem se encuentra en la clase NP**.

4. Conclusiones

Acá irían las conclusiones de todo nuestro trabajo :)